

Maritime CargoService: Sprint 2

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 20, 2026

1. Introduction

Il punto di partenza di questo sprint è il prototipo realizzato nello Sprint 1, documentato al seguente [link](#)^o.

Lo Sprint 1 ha prodotto un prototipo eseguibile del **cargoservice** in cui il ciclo principale di carico viene gestito tramite attori QAK, con componenti simulati per IOPort, sonar, LED e markerdevice, e con il **cargorobot** modellato come wrapper del servizio **robotsmart26**.

L'architettura finale dello Sprint 1, riportata di seguito, costituisce il riferimento di partenza per lo Sprint 2.

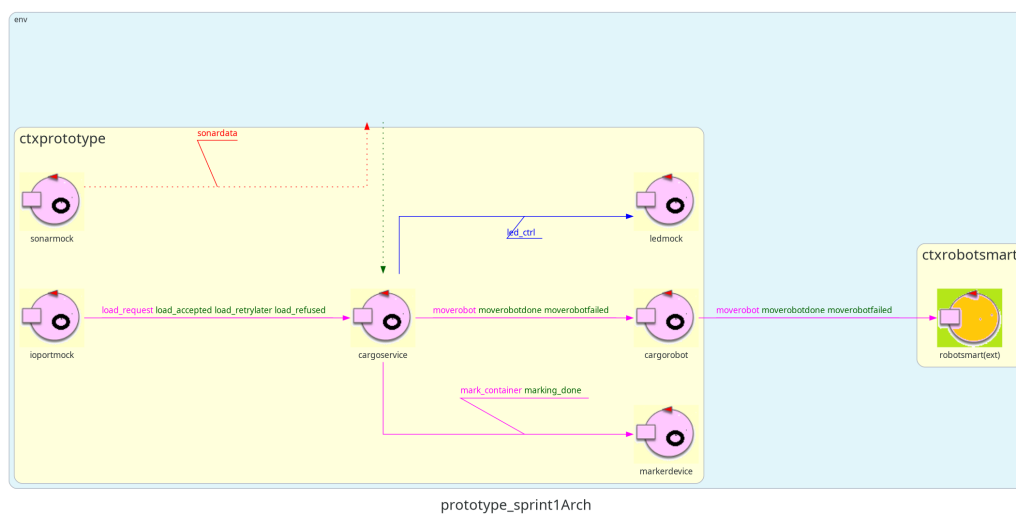


Figure 1: Architettura finale definita nello Sprint 1.

1.1. Goal dello Sprint 2

Il goal dello Sprint 2 è quello definito nella pagina di sintesi dello Sprint 1.

L'obiettivo dello Sprint 2 è evolvere il prototipo sviluppato sostituendo progressivamente i componenti simulati con le rispettive implementazioni reali:

- IOPort come Web GUI;
- sonar e LED collegati al PicoW;
- completare la gestione dello stato **Out of service** integrando il sonar reale, implementando la verifica della misura e l'interruzione/ripresa del movimento del cargorobot.

Si stima una durata di circa **30 ore** complessive di lavoro, corrispondenti a circa 10 ore per ciascun componente del gruppo.

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o.

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Gli slot1-4 rappresentano le aree della hold riservate per immagazzinare ciascuno un container.
- Lo slot5 rappresenta un'area in cui il cargorobot deve temporaneamente depositare un container, prima di posizionarlo in uno degli slot1-4. Durante la sosta temporanea, un dispositivo 'marker' etichetta il container con un codice a barre identificativo e segnala quando l'attività di marcatura è completata.
- L'IOPort è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e lo stato attuale della hold.
- Il sensore associato all'IOPort è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D , tale che $D < D_{\text{FREE}}/2$, per un tempo ragionevole (ad esempio 3 secondi).
- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold;
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente;
 - il messaggio "**Out of service**" se il sensore sonar misura la distanza del container dal sonar stesso $D > D_{\text{FREE}}$ per almeno 3 secondi.

3. Requirement analysis

L'analisi dei requisiti è stata svolta nello [Sprint 0°](#) e approfondita nello [Sprint 1°](#). Tali risultati vengono assunti come validi anche per lo Sprint 2.

4. Problem analysis

Come definito precedentemente, bisogna sostituire progressivamente i componenti simulati con componenti accessibili attraverso tecnologie compatibili con la loro natura.

Nota: Il nostro team di sviluppo ha utilizzato una scheda **ESP32** invece del **Raspberry Pi Pico W** indicato dalla committente. L'architettura resta sostanzialmente invariata: anche ESP32 ha connettività ad internet, supporta MicroPython, ha un LED, dispone dei GPIO necessari per il sensore HC-SR04 e offre prestazioni generalmente superiori.

L'unica differenza riguarda aspetti implementativi, come la diversa numerazione dei pin e l'assenza di moduli specifici del Pico W (ad esempio rp2). Tuttavia, tali differenze non incidono sul progetto.

4.1. Osservabilità dello stato della Hold e considerazioni sulla Web GUI

Nello Sprint 1, l'IOPort era modellato come un attore QAK “mock” all'interno dello stesso contesto dell'hold. Questa vicinanza tecnologica permetteva all'IOPort di reagire direttamente ai singoli eventi e messaggi asincroni scambiati sulla rete di attori.

Adesso l'IOPort evolve in una Web GUI eseguita in un browser web esterno. Questa transizione fisica e architetturale solleva una problematica fondamentale che rende i messaggi QAK nativi inadeguati per l'aggiornamento dell'interfaccia:

Se infatti il modello QAK descrive l'evoluzione del sistema tramite **messaggi**, una Web GUI deve invece rappresentare lo stato corrente del sistema in un **determinato istante**. Ricostruire tale stato elaborando l'intera sequenza dei messaggi risulterebbe complesso e renderebbe il frontend dipendente dalla logica degli attori. È quindi necessario introdurre una rappresentazione esplicita e serializzata dello stato del sistema, così da lasciare alla GUI l'unico ruolo di associare i dati ricevuti sui componenti grafici. Il **cargoservice** continuerà a gestire le transizioni di stato.

Bisogna considerare lo **stato della Hold** come una vera e propria risorsa del sistema, mantenuta dal cargoservice e resa disponibile ai componenti esterni. Questa soluzione separa chiaramente responsabilità e livelli di astrazione: il cargoservice continua ad essere l'unico responsabile dell'evoluzione dello stato, mentre la GUI si limita esclusivamente alla sua rappresentazione grafica.

Lo stato dinamico della **Hold** e del sistema viene astratto e centralizzato in una rappresentazione serializzabile in formato **JSON** (mediante il metodo `toJson`), indipendente

dal linguaggio di programmazione e direttamente interpretabile dal motore del client da noi scelto, ovvero JavaScript.

```
public static String toJson(String serviceState, String workingState, boolean
ioPortOccupied, int reservedSlot) {
    return INSTANCE.doToJson(serviceState, workingState, ioPortOccupied,
reservedSlot);
}
```

Esempio di JSON creato dal metodo `toJson` :

```
{
  "serviceState": "engaged",           // engaged, disengaged
  "workingState": "Service working",   // Service working, Out of service
  "ioPortOccupied": true,              // true o false
  "reservedSlot": 2,                  // slot riservato per l'operazione
  "slots": {                          // occupied, reserved, free
    "slot1": "occupied",
    "slot2": "reserved",
    "slot3": "free",
    "slot4": "occupied"
  }
}
```

Il codice della classe `Hold` si trova al seguente [link](#)^o.

Il `cargoservice` si assume la responsabilità di rigenerare questa stringa JSON e di notificarla all'esterno ogni volta che si verifica una variazione significativa del sistema (es. transizione tra gli stati engaged/disengaged, variazione delle distanze del sonar, ingresso/uscita dallo stato Out of service o completamento del deposito).

4.2. Protocolli per l'esposizione dello stato

Una volta stabilito che lo stato del `cargoservice` e della `Hold` debbano essere rappresentati come una risorsa osservabile, occorre individuare il protocollo più adatto per renderla disponibile all'esterno del sistema.

La Web GUI deve infatti poter ricevere gli aggiornamenti quando esso cambia.

Le principali alternative sono:

1. Utilizzare il protocollo **HTTP tradizionale**, interrogando periodicamente il `cargoservice` mediante richieste GET (polling). Tale soluzione risulta però inefficiente, poiché genera richieste anche quando lo stato del sistema rimane invariato, aumentando il traffico di rete e il carico sui componenti coinvolti.
2. Utilizzare il protocollo **MQTT** pubblicando un messaggio ogni volta che lo stato cambia. MQTT, nonostante permetta notifiche push, richiede di modellare lo stato come una sequenza di pubblicazioni su topic. Tale approccio risulta meno naturale

poiché lo stato della `Hold` rappresenta una risorsa persistente piuttosto che una successione di eventi.

3. Utilizzare il protocollo **CoAP** sfruttando l'estensione `Observe` già supportato dal runtime QAK. Questo approccio risulta particolarmente adatto poiché il runtime QAK rende gli attori risorse CoAP nativamente “osservabili”. `Observe` è un'estensione del protocollo CoAP che permette una comunicazione asincrona (di tipo `publish-subscribe`) permettendo al nostro web server di osservare gli attori venendo notificato (tramite la funzione nativa di QAK `updateResource`) ogni volta che lo stato cambia (senza quindi dover fare `polling`). Il **cargoservice** può quindi limitarsi ad aggiornare la propria risorsa ogni volta che lo stato cambia, i client si “iscriveranno” ad essa ricevendo automaticamente la nuova rappresentazione. Si sceglie pertanto la seguente soluzione.

Il **cargoservice** rende quindi disponibile il proprio stato sotto forma di documento JSON, aggiornando la risorsa (unicamente quando avviene una variazione significativa) tramite la primitiva `updateResource(...)`.

Esempio: transizione stato `engaged/disengaged`

```
State disengaged {
  println("cargoservice | DISENGAGED: waiting for requests...") color blue
  [# val statusJson = Hold.toJson(CargoState, if(ServiceWorking) "Service
working" else "Out of service", IOPortOccupied, ReservedSlotId) #]
  updateResource [# statusJson #]
}
// ...
State engaged {
  println("cargoservice | ENGAGED: evaluating next step...") color blue
  [# val statusJson = Hold.toJson(CargoState, if(ServiceWorking) "Service
working" else "Out of service", IOPortOccupied, ReservedSlotId) #]
  updateResource [# statusJson #]
}
Goto do_robot_job if [# IOPortOccupied && CargoState == "engaged" &&
ServiceWorking #] else wait_for_container
```

Il codice completo dell'attore **cargoservice** è disponibile al seguente [link](#)^o.

Rimane da risolvere il problema della comunicazione tra il mondo QAK e il browser.

Una prima possibilità sarebbe consentire alla Web GUI di osservare direttamente la risorsa CoAP del **cargoservice**.

Tale soluzione non risulta però praticabile poiché i browser moderni non supportano nativamente il protocollo CoAP.

Una seconda possibilità consiste nell'introdurre un componente intermedio che osservi la risorsa CoAP e renda disponibili gli aggiornamenti mediante protocolli compatibili con il browser, dando inoltre il vantaggio di mantenere tutta la logica di integrazione in un unico componente, lasciando indipendenti sia il **cargoservice** sia la GUI.

Viene perciò introdotto un server chiamato **ioport-backend** con il compito di:

- osservare la risorsa CoAP del **cargoservice**, ricevendo gli aggiornamenti in formato JSON tramite il meccanismo **Observe**;
- esporre tali informazioni alla Web GUI tramite un protocollo la cui scelta è definita nella sezione successiva;
- inoltrare al **cargoservice** le richieste provenienti dalla GUI.

4.3. Realizzazione dell'IOPort come Web GUI

L'IOPort richiesto dalla committente è costituito logicamente da:

- un pushbutton, utilizzato dal cliente per inviare una richiesta di carico di un container;
- un display, utilizzato per mostrare la risposta alla richiesta e lo stato corrente della hold.

La sua implementazione viene suddivisa in due parti:

- una pagina web eseguita nel browser (disponibile al seguente [link](#)^o). Si utilizza il motore JavaScript nativo per la gestione della logica di interazione con l'utente e per la visualizzazione dello stato del sistema. I dati verranno visualizzati su una pagina HTML.
- un server intermediario (IOPort backend) che collega il browser al sistema QAK (disponibile al seguente [link](#)^o) e che funge da web server per la pagina web. Si è scelto di utilizzare Javalin, un framework leggero per la creazione di web server in Java, che permette di gestire facilmente le richieste HTTP. Per implementare il meccanismo di Observe di CoAP si utilizzano specifiche funzioni di libreria (il codice della gestione dell'Observer si trova al seguente [link](#)^o).

4.3.1. Invio della load_request

Quando il cliente preme il pulsante, la GUI invia una richiesta HTTP **POST** al server intermediario.

Il server traduce tale richiesta in una **load_request** indirizzata al **cargoservice** e attende una delle risposte già definite nello Sprint 0 (il codice della gestione della **load_request** si trova al seguente [link](#)^o):

La risposta viene quindi restituita alla GUI e mostrata sul display.

Questa soluzione mantiene invariato il protocollo applicativo QAK già validato nello Sprint 1: HTTP viene utilizzato soltanto nel tratto browser-server, mentre l'interazione con il **cargoservice** continua a essere espressa mediante i messaggi del modello.

4.3.2. Aggiornamento del display

Per effettuare l'aggiornamento dello stato sul display si potrebbe:

1. utilizzare un polling periodico da parte della GUI, che interroga il server per verificare eventuali variazioni dello stato, anche in questo caso, notoriamente pesante come meccanismo;
2. utilizzare WebSocket, che realizza invece una comunicazione bidirezionale permanente, permettendo al server di notificare immediatamente ogni aggiornamento ricevuto tramite CoAP Observe.

Si sceglie pertanto tale soluzione in quanto più efficiente e reattiva.

In questo modo il server, dopo aver ricevuto l'aggiornamento dello stato del **cargoservice** può inoltrare immediatamente le informazioni alla GUI senza che quest'ultima debba richiederle periodicamente (il codice della gestione del WebSocket si trova al seguente [link](#)^o).

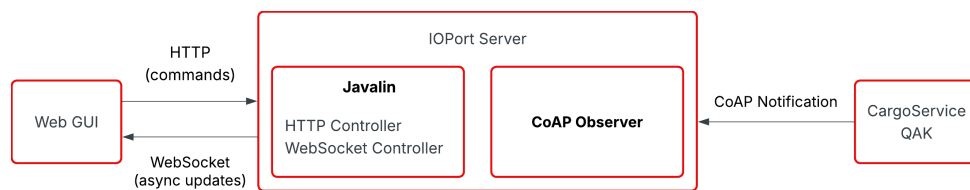


Figure 2: Architettura dell'IOPort server

4.3.3. Evoluzione dell'IOPort

Nei primi sprint l'IOPort era stata modellata come un attore QAK esclusivamente per simulare il comportamento dell'interfaccia utente durante la prototipazione del sistema, verificando velocemente l'interazione con il cargoservice. Con l'introduzione della Web GUI questa modellazione non risulta più necessaria. Il precedente attore QAK viene perciò rimosso e sostituito da una architettura client-server composta da frontend e backend (il quale interagisce con il cargoservice).

Il backend assume il ruolo di adattatore di protocollo (bridge): traduce le richieste HTTP provenienti dalla GUI nelle corrispondenti Request QAK verso il cargoservice (via TCP), e inoltra al browser, tramite WebSocket, gli aggiornamenti di stato ricevuti dal cargoservice mediante CoAP Observe.

Il contesto "ctxioport" rappresentava il nodo di esecuzione dell'attore ioport ma, poichè ora l'interfaccia utente è realizzata come applicazione web esterna al sistema QAK, essa non necessita più di un context dedicato. Il backend IOPort costituisce un processo separato che comunica con il sistema mediante protocolli standard: HTTP, WebSocket e CoAP.

4.4. Integrazione del sonar reale

Nello sprint precedente, il sonar era stato modellato tramite l'attore `sonarmock`, il quale simulava il comportamento del dispositivo fisico generando eventi di tipo `sonardata` direttamente indirizzati al `cargoservice`.

Con l'introduzione del sonar reale, collegato a un dispositivo ESP32, questo approccio non è più applicabile. Lo script eseguito sull'ESP32 ha infatti il solo compito di acquisire

periodicamente la distanza rilevata dal sensore e rendere disponibili tali informazioni al sistema distribuito.

È quindi necessario adottare un protocollo che garantisca leggerezza, semplicità di integrazione e interoperabilità tra componenti eterogenei. Per questi motivi viene scelto MQTT, un protocollo basato sul modello publish/subscribe, particolarmente adatto a dispositivi con risorse limitate e scenari IoT.

Le misurazioni rilevate dal sonar vengono quindi pubblicate dall'ESP32 su un topic MQTT dedicato, al quale i componenti interessati del sistema possono sottoscrivere per ricevere gli aggiornamenti in modo disaccoppiato rispetto al dispositivo fisico.

Per far ricevere ora i messaggi al cargoservice le possibilità sono due:

1. modificare il cargoservice affinché riceva direttamente messaggi MQTT. Questa soluzione renderebbe però il cargoservice dipendente dal protocollo di comunicazione utilizzato dal dispositivo fisico, riducendone la riusabilità e introducendo dettagli infrastrutturali nella logica applicativa
2. Si introduce quindi un **adapter**, un componente dedicato (**sonaradapter**) che svolge esclusivamente il compito di integrazione tra il dispositivo fisico e il sistema esistente. Esso, similmente al mock riceve le misurazioni pubblicate dall'ESP32 tramite MQTT, converte il payload ricevuto nel formato previsto dal modello e inoltra le informazioni al cargoservice mediante il dispatch `incoming_sonar`.

In questo modo il cargoservice continua a ricevere esclusivamente messaggi QAK e rimane completamente indipendente dal protocollo utilizzato dal dispositivo fisico.

Il codice del sonar può essere trovato al seguente [link](#).

```
QActor sonaradapter context ctxdevices {
  State s0 initial {
    subscribe "sonardata"
    println("sonaradapter | STARTED - Listening to MQTT event
wall_sonardata on topic: sonardata") color green
  }
  Goto work

  State work {}
  Transition t0
    whenEvent wall_sonardata → handle_sonar_payload

  State handle_sonar_payload {
    onMsg(wall_sonardata : distance(D)) {
      [#
        val Distance = payloadArg(0)
      #]
      println("sonaradapter | Forwarding incoming_sonar($Distance) to
cargoservice") color cyan
      forward cargoservice -m incoming_sonar : distance($Distance)
    }
  }
}
```

```
} Goto work
```

??? PERCHE USI
AMO EVENT PER RICEZIONE INVIO MQTT E DISPATCH PER INVIO DATI
TRA ATTORI ??

Si è scelto di distinguere i due messaggi, separando la comunicazione con il dispositivo fisico dalla comunicazione interna al sistema software:

```
Event wall_sonardata
Dispatch incoming_sonar
```

`wall_sonardata` rappresenta il dato ricevuto dal sonar fisico tramite MQTT e quindi appartiene al livello di comunicazione esterno.

`incoming_sonar` rappresenta invece il messaggio interno utilizzato dagli attori QAK per propagare la misura della distanza, mantenendo separata la logica applicativa dai dettagli del protocollo MQTT.

Il codice del cargoservice che riceve il messaggio `incoming_sonar` e aggiorna lo stato della risorsa CoAP è riportato di seguito.

```
State handle_sonar {
    onMsg(incoming_sonar : distance(D)) {
        [#
            ...
            val statusJson = Hold.toJson(CargoState, if(ServiceWorking)
"Service working" else "Out of service", IOPortOccupied, ReservedSlotId)

            #]
            updateResource [# statusJson #]
        }
    }
}

Goto do_robot_job if [# IOPortOccupied && CargoState = "engaged" &&
ServiceWorking #] else returnToState
```

Il codice del cargoservice che gestisce l'evento `incoming_sonar` è disponibile al seguente [link](#)^o).

4.5. Validazione temporale delle misure sonar

Da requisiti, una singola misura del sonar non comporta necessariamente una transizione dello stato. Sia la **presenza del container** sia la condizione di **Out of service** devono essere riconosciute solo quando la relativa condizione permane per almeno **tre secondi** consecutivi.

Dal punto di vista applicativo è quindi necessario distinguere tre intervalli di misura:

1. `D < DFREE / 2` che indica una possibile presenza del container davanti all'TOPort;

2. `D > DFREE` che indica una possibile condizione di malfunzionamento del sistema di carico (Out of service);
3. valori intermedi, che non soddisfano nessuna delle due condizioni precedenti.

Una possibile soluzione sarebbe demandare questa validazione direttamente all'ESP32, facendogli pubblicare solamente eventi già validati temporalmente. In questo modo però l'accoppiamento del dispositivo con il dominio applicativo aumenta notevolmente.

Si preferisce pertanto mantenere quindi l'ESP32 una semplice componente di acquisizione dati. Sarà quindi il cargoservice a dover memorizzare:

1. l'istante in cui viene rilevata per la prima volta la condizione $D < DFREE/2$;
2. l'istante in cui viene rilevata per la prima volta la condizione $D > DFREE$;
3. l'annullamento del conteggio quando la misura torna nell'intervallo normale.

Solo quando una delle due condizioni permane per almeno tre secondi consecutivi viene effettivamente effettuata una transizione di stato (Il codice responsabile della gestione delle transizioni di stato è riportato al seguente [link](#)^o).

```
...
if (Dist < DFreeDiv2) {
  if (ContainerPendingStart < 0L) {
    ContainerPendingStart = Now
  } else if (Now - ContainerPendingStart ≥ 3000L) {
    IOPortOccupied = true
    ContainerPendingStart = -1L
  }
} else {
  ContainerPendingStart = -1L
  IOPortOccupied = false
}

if (Dist > DFree) {
  if (ServiceWorking) {
    if (OutOfServicePendingStart < 0L) {
      OutOfServicePendingStart = Now
    } else if (Now - OutOfServicePendingStart ≥ 3000L) {
      ServiceWorking = false
      OutOfServicePendingStart = -1L
      println("cargoservice | Sonar D>DFREE ($Dist > $DFree) sostenuto
per 3s → OUT OF SERVICE!")
      forward("stop_robot", "stop(none)", "cargorobot")
    }
  }
} else {
  OutOfServicePendingStart = -1L
  if (!ServiceWorking) {
    ServiceWorking = true
    println("cargoservice | Sonar D≤DFREE ($Dist ≤ $DFree) → SERVICE
WORKING again!")
    forward("resume_robot", "resume(none)", "cargorobot")
  }
}
```

```
}  
...
```

Il timer non viene riavviato ad ogni misura. Viene registrato solamente il primo istante in cui la condizione diventa vera.

Se una misura interrompe la continuità della condizione, il conteggio viene annullato e dovrà eventualmente ripartire dalla misura successiva.

Lo stesso identico schema vale per lo stato Out of service, sostituendo la condizione $D > DFREE$ e la variabile che memorizza il tempo di inizio del possibile guasto.

4.6. Integrazione del LED reale

Anche il LED costituisce un dispositivo fisico e pone il medesimo problema di integrazione affrontato per il sonar.

La soluzione più valida resta, come nel sonar, l'introduzione di un **adapter** che traduca i comandi logici nel protocollo MQTT, evitando che sia il cargoservice a controllare direttamente il dispositivo.

Nello Sprint 1 il **cargoservice** inviava a **ledmock** il dispatch che rappresentava le operazioni logiche di **off** e **blink**. Esso viene mantenuto come interfaccia logica utilizzata da cargoservice.

L'adapter riceve il comando e lo pubblica su un topic MQTT dedicato al LED. Il software sull'ESP32 si sottoscrive al topic e traduce il valore ricevuto nell'operazione hardware corrispondente.

```
Dispatch led_ctrl : ledCmd(CMD)  
Event    led_event    : ledCmd(CMD)  
  
QActor ledadapter context ctxdevices {  
  
    State s0 initial {  
        println("ledadapter | STARTED - Routing dispatches to MQTT events on  
topic: leddata") color green  
    }  
    Goto work  
  
    State work { }  
    Transition t0  
        whenMsg led_ctrl → handle_led_cmd  
  
    State handle_led_cmd {  
        onMsg(led_ctrl : ledCmd(CMD)) {  
            [# val CMD = payloadArg(0) #]  
            println("ledadapter | Publishing LED command on MQTT: $CMD")  
color blue  
            emit led_event : ledCmd($CMD)  
        }  
    }  
}
```

```

    Goto work
}

```

Il **cargoservice** interagisce con l'attore esterno in questo modo:

Accensione del LED dopo l'accettazione della richiesta:

```

ReservedSlotId = SlotId
CargoState = "engaged"

forward ledadapter -m led_ctrl : ledCmd(blink)
[# val SlotName = "slot$ReservedSlotId" #]
replyTo load_request with load_accepted : loadAccepted($SlotName)

```

Spegnimento del LED al completamento del servizio:

```

State finish_job {
    ...
    [#
        CargoState = "disengaged"
        ReservedSlotId = -1
    #]
    forward ledadapter -m led_ctrl : ledCmd(off)
    ...
}

```

Spegnimento del LED in caso di errore durante il ritorno alla Home:

```

State handle_home_fail {
    ...
    [#
        CargoState = "disengaged"
        ReservedSlotId = -1
    #]
    forward ledadapter -m led_ctrl : ledCmd(off)
    ...
}

```

Spegnimento del LED per timeout del deposito:

```

State handle_deposit_timeout {
    if [# CargoState == "engaged" && !IOPortOccupied #] {
        ...
        [#
            Hold.freeSlot(ReservedSlotId)
            CargoState = "disengaged"
            ReservedSlotId = -1
        #]
        forward ledadapter -m led_ctrl : ledCmd(off)
        ...
    }
}

```

```
}  
}
```

4.7. Gestione dello stato **Out of service**

Nello Sprint 1 il **cargoservice** aggiornava la variabile `ServiceWorking` sulla base degli eventi sonar, ma non interrompeva un movimento già in corso.

Nello Sprint 2 il comportamento viene completato in accordo con il chiarimento fornito dalla committente:

- quando la condizione `D > D_FREE` persiste per almeno tre secondi, il sistema entra nello stato **Out of service**;
- durante tale stato non vengono accettate nuove richieste;
- se il robot è in movimento, il piano deve essere interrotto;
- quando il sonar torna a misurare `D ≤ D_FREE`, il sistema ritorna nello stato **Service working**;
- se un movimento era stato interrotto, deve essere ripreso.

La gestione richiede di distinguere almeno due situazioni:

1. il sistema entra in **Out of service** mentre non è in corso alcun movimento;
2. il sistema entra in **Out of service** durante l'esecuzione di una procedura di movimentazione.

Nel primo caso è sufficiente aggiornare lo stato operativo e impedire l'accettazione di nuove richieste.

Nel secondo caso il **cargoservice** deve inoltre richiedere l'interruzione del movimento e ricordare che esiste una procedura sospesa. Al ripristino del servizio, il movimento viene ripreso secondo le operazioni offerte da **RobotSmart26**.

L'ingresso nello stato **Out of service** non deve:

- liberare automaticamente lo slot riservato;
- riportare il sistema nello stato `disengaged`;
- annullare la procedura di carico.

Si tratta infatti di una sospensione temporanea e non del fallimento definitivo dell'operazione.

4.8. Gestione coerente dello stato della hold

La prenotazione di uno slot e la sua effettiva occupazione rappresentano due momenti distinti.

Quando una `load_request` viene accettata, lo slot viene riservato affinché non possa essere assegnato a un'altra richiesta. Esso può però essere considerato fisicamente occupato soltanto dopo che il robot ha completato il deposito.

È quindi opportuno distinguere logicamente almeno i seguenti stati:

- libero;
- riservato;
- occupato.

Questa distinzione permette alla Web GUI di mostrare uno stato più fedele della hold e rende esplicito cosa debba accadere nei casi di timeout, sospensione o fallimento del movimento.

In particolare:

- in caso di timeout prima del deposito, lo slot riservato torna libero;
- durante una sospensione **Out of service**, lo slot rimane riservato;
- dopo il deposito completato, lo slot diventa occupato;
- in caso di fallimento del robot, lo slot non viene liberato automaticamente.

4.9. Evoluzione dell'architettura

Rispetto allo Sprint 1 vengono introdotti i seguenti elementi:

- una Web GUI che realizza l'IOCome posso evolvere lo sprint1 naturalmente nello sprint2?Port;
- un server intermediario per HTTP, WebSocket e osservazione CoAP;
- un broker MQTT per la comunicazione con il PicoW;
- il software sul PicoW per il sonar e il LED;
- un componente di adattamento tra MQTT e i messaggi QAK;
- la pubblicazione dello stato del **cargoservice** come risorsa CoAP osservabile.

Il **cargoservice** rimane l'orchestratore del ciclo di carico e la **Hold** rimane la sorgente dello stato degli slot. I nuovi componenti non trasferiscono altrove la logica applicativa, ma rendono possibile l'interazione con browser e dispositivi fisici.

L'evoluzione preserva quindi le interfacce logiche introdotte nello Sprint 1:

- `load_request` e relative reply per l'IOPort;
- `sonardata` per le misurazioni del sonar;
- `led_ctrl` per il controllo del LED;
- `moverobot` per la movimentazione;
- `mark_container` per la marcatura.

Le principali modifiche riguardano il trasporto dei messaggi e l'osservabilità dello stato, non il significato delle interazioni applicative già validate.

5. Project

5.1. Struttura del progetto

Nota: Da completare con la struttura finale dei moduli e con i riferimenti ai sorgenti implementati nello Sprint 2.

5.2. Implementazione dell'IOPort

Nota: Da completare.

5.3. Implementazione dei dispositivi reali

Nota: Da completare.

5.4. Implementazione della gestione Out of service

Nota: Da completare.

6. Test plans

Nota: Da completare con i test funzionali e di integrazione previsti per lo Sprint 2.

6.1. Test IOPort

Nota: Da completare.

6.2. Test sonar e LED

Nota: Da completare.

6.3. Test Out of service e ripresa del robot

Nota: Da completare.

7. Deployment

Al fine di semplificare la manutenzione, la distribuzione e garantire modularità e isolamento dei componenti, l'architettura del sistema è stata progettata seguendo un approccio a microservizi. Si è scelto di distribuirli come container **Docker**, mentre l'intero processo di avvio ed esecuzione è orchestrato tramite **Docker Compose**.

Il sistema prevede l'esecuzione di alcuni servizi infrastrutturali e di integrazione con componenti esterni (quali **mosquitto**, il broker MQTT, e **wenv**, **RobotSmart26** e **RobotOutGui25** forniti per il controllo di base del robot e per la visualizzazione).

Si prevede poi l'avvio dei seguenti componenti (descritti nelle fasi precedenti del documento) containerizzati: **cargoservice**, **cargorobot**, **devices** e **ioport-backend**

Per semplificare l'avvio dell'intero sistema è stato predisposto uno script Bash (`start.sh`) che automatizza le operazioni di compilazione, costruzione delle immagini Docker e avvio dei container.

Lo script esegue automaticamente le seguenti operazioni (eseguibili anche manualmente):

1. Accede alle directory dei microservizi ed esegue il comando

```
./gradlew distTar
```

per compilare il progetto e generare l'archivio contenente la distribuzione dell'applicazione.

2. Crea la rete Docker interna se non esiste.
3. Costruisce le immagini utilizzando gli archivi generati nella fase precedente e avvia tutti i container in modalità detached.

```
docker compose up --build -d
```

Al termine dell'avvio, le interfacce grafiche del sistema sono accessibili dal browser ai seguenti indirizzi:

- **WebGui dell'IOPort** disponibile sulla porta locale 8086
- **Ambiente grafico del robot** disponibile sulla porta locale 8090

8. Maintenance

Nota: Da completare con eventuali note di manutenzione, limiti noti e attività rinviate agli sprint successivi.

9. Pagina di sintesi

9.1. Architettura finale dello Sprint 2

Nota: Da completare con l'architettura finale prodotta nello Sprint 2.

9.2. Goal dello sprint successivo

Nota: Da completare al termine dello Sprint 2.

10. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340